> For my thread to wait for a python daemon thread to shutdown, do I need to join the main thread?

No, you don't join the main thread — you join the **daemon thread** you want to wait for. A thread joins the thread it's waiting on, so the thread that needs to wait calls `.join()` on the daemon thread.

Key points:

1. **Daemon threads** are killed abruptly when the main thread exits. If you want them to finish cleanly, you must `join()` them from some non-daemon thread before the program ends.
2. **Only non-daemon threads can join daemons meaningfully.** The main thread is itself non-daemon, so it can (and typically does) call `daemon_thread.join()` before exiting. A daemon thread can also join another daemon thread.
3. **You cannot join the main thread from a daemon thread to keep it alive** — the main thread doesn't wait on daemons by design. Once `main` returns and all non-daemon threads finish, the interpreter tears down daemons regardless.

Example:

```python
import threading, time

def worker():
    time.sleep(2)
    print("daemon done")

t = threading.Thread(target=worker, daemon=True)
t.start()

t.join()   # main thread waits for the daemon
print("main exiting")
```

If you have *another* thread that needs to wait for that daemon, that other thread calls `t.join()`, not `main_thread.join()`.

One caveat: `threading.main_thread().join()` from another thread is allowed but only useful to wait until `main` finishes — it won't help daemons survive, since daemons die

at interpreter shutdown anyway.

So: **join the daemon thread from whichever thread needs to wait; never join the main thread for this purpose.**

Will this test work?

```
def test_01_obey_shutdown_onwait(self):
    self.server.start()
    self.shutdown.set()
    try:
        self.server.join(timeout=3)
    except TimeoutError:
        self.fail()
```

No, this test will not work as written. There are two problems:

1. `Thread.join()` doesn't raise `TimeoutError`

`join(timeout=3)` returns `None` whether or not the thread finished. If the timeout expires, the thread is simply *still alive* — no exception is raised. So your `try/except TimeoutError` will never catch anything, and the test will pass even if the thread hangs.

You need to check `is_alive()` after the join:

```python
self.server.join(timeout=3)
self.assertFalse(self.server.is_alive(), "server did not shut down within 3s")
```

2. If `server` is a daemon thread, this test may pass trivially or be misleading

Two sub-issues here:

- **If `server` is a daemon thread**, it will be killed at interpreter exit anyway, so "shutdown" isn't really being verified — you're just checking that it noticed the `shutdown` event and returned within 3 seconds. That's a valid thing to test, but only if `server` 's loop actually checks `self.shutdown` and returns. Make sure the worker loop is doing something like:

```python
while not self.shutdown.is_set():
    ...work...
```

Otherwise `join` will just time out.

- **Timing/fragility**: you set `shutdown` *immediately* after `start()`, which can race with the thread even entering its loop. That's usually fine, but if you want to test "obey shutdown *on wait*" specifically (i.e., it's blocked in a wait when shutdown fires), you likely want the thread to be *in* the wait before you set the event — otherwise you're testing a different path (shutdown set before the loop starts).

A corrected version

```python
def test_01_obey_shutdown_onwait(self):
    self.server.start()
    # optionally: wait until server is actually blocked in its wait
    # e.g. poll a "ready" event the server sets before waiting
    self.shutdown.set()
    self.server.join(timeout=3)
    self.assertFalse(self.server.is_alive(), "server did not shut down within 3s")
```

Summary

- `join(timeout=...)` never raises `TimeoutError` → your `except` is dead code.
- Use `assertFalse(thread.is_alive())` instead.
- If `server` is a daemon, the test only verifies cooperative shutdown (the loop returned), not survival — which is fine, but be clear about what you're asserting.
- Consider synchronizing so the thread is genuinely *waiting* when `shutdown` is set, if that's the behavior under test.